

MedIntel

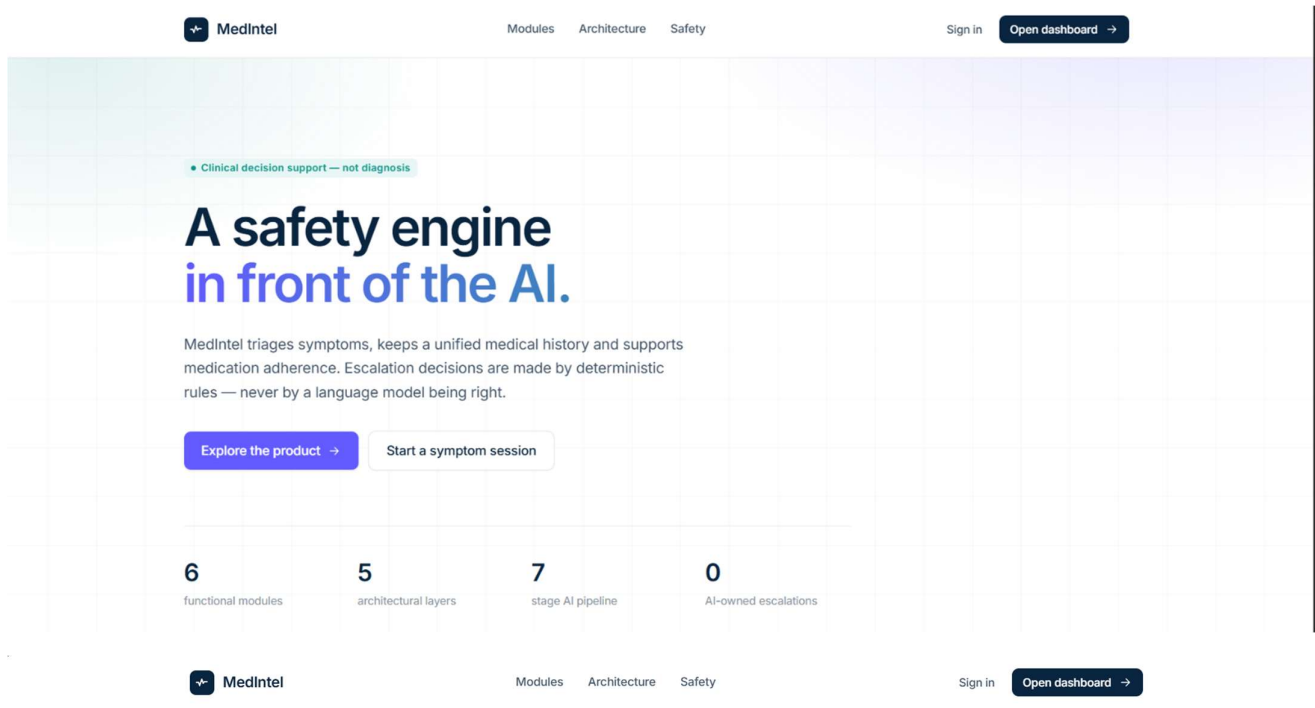
User Interface Design and Implementation

Rahul KT (2024BCS0224)

Sathwik Teja E (2024BCD0040)

Aryan Gupta (2024BCS0020)

Sunil (2024BCS0340)



SCOPE — VERSION 1.0

Six modules, one product surface.

01 Authentication & profile JWT sessions, hashed credentials, and a profile that carries age, allergies and chronic conditions into every reasoning step.	02 Symptom analysis Structured intake passes through the red-flag rules engine before the AI is ever called. Output is a ranked possibility list, never a verdict.	03 AI health chat Follow-up conversation that remembers the session context and the stored history — with the same guardrails applied to every turn.
04 Report upload PDF and image upload, OCR extraction off the request path, and a plain-language summary of key values attached to the timeline.	05 Medicine reminders Dose schedules driven by recurring jobs, delivered by push or e-mail, with a taken-or-missed adherence log.	06 Medical history One chronological timeline of sessions, reports and medication — exportable for a real consultation.

Purpose: Introduces the core scope and six architectural modules that make up the MedIntel platform.

Main Components: Navigation bar (Modules, Architecture, Safety), a 6-card grid outlining Authentication & Profile, Symptom Analysis, AI Health Chat, Report Upload, Medicine Reminders, and Medical History.

Buttons/Actions: Sign in, Open dashboard →.

Functionality: Informs prospective users and evaluators about the modular architecture and end-to-end scope of the application before entry.

MedIntel

ModulesArchitectureSafety

Sign inOpen dashboard →

LOGICAL VIEW

Layered, and strictly so.

A modular monolith with two rules that make the style real: a layer calls only the layer directly beneath it, and no layer may be skipped. The React client cannot reach MongoDB — It does not know MongoDB exists.

- ✓ Cross-cutting concerns run vertically as the Express middleware chain.
- ✓ The AI provider sits behind an adapter — swapped by config, never by code.
- ✓ Clean module seams keep the AI service extractable into its own process later.

Presentation

React SPA

API / Gateway

Express · auth, validation, rate limit

Business logic

Triage rules · orchestrator · scheduler

AI service

Adapter · redaction · validator · breaker

Data access

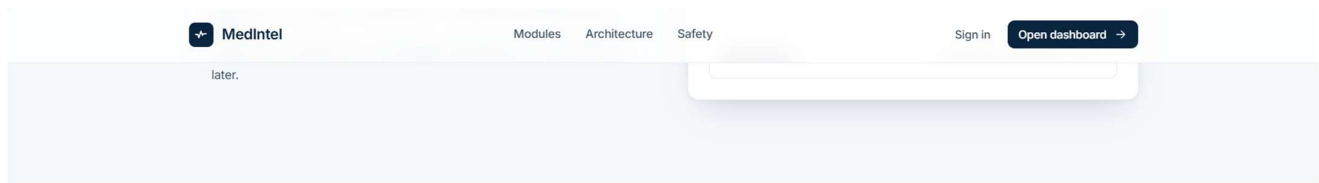
Repositories · Mongoose · Redis

Purpose: Explains the layered software architecture and separation of concerns powering the backend.

Main Components: Layer diagram illustrating the strict downward-calling stack: *Presentation (React SPA)* → *API / Gateway (Express)* → *Business Logic (Triage rules, Orchestrator)* → *AI Service (Adapter, Redaction)* → *Data Access (Repositories, Mongoose, Redis)*.

Buttons/Actions: Sign in, Open dashboard →.

Functionality: Communicates system design principles, showing that AI providers are abstracted behind adapters and client-side code has no direct database access.



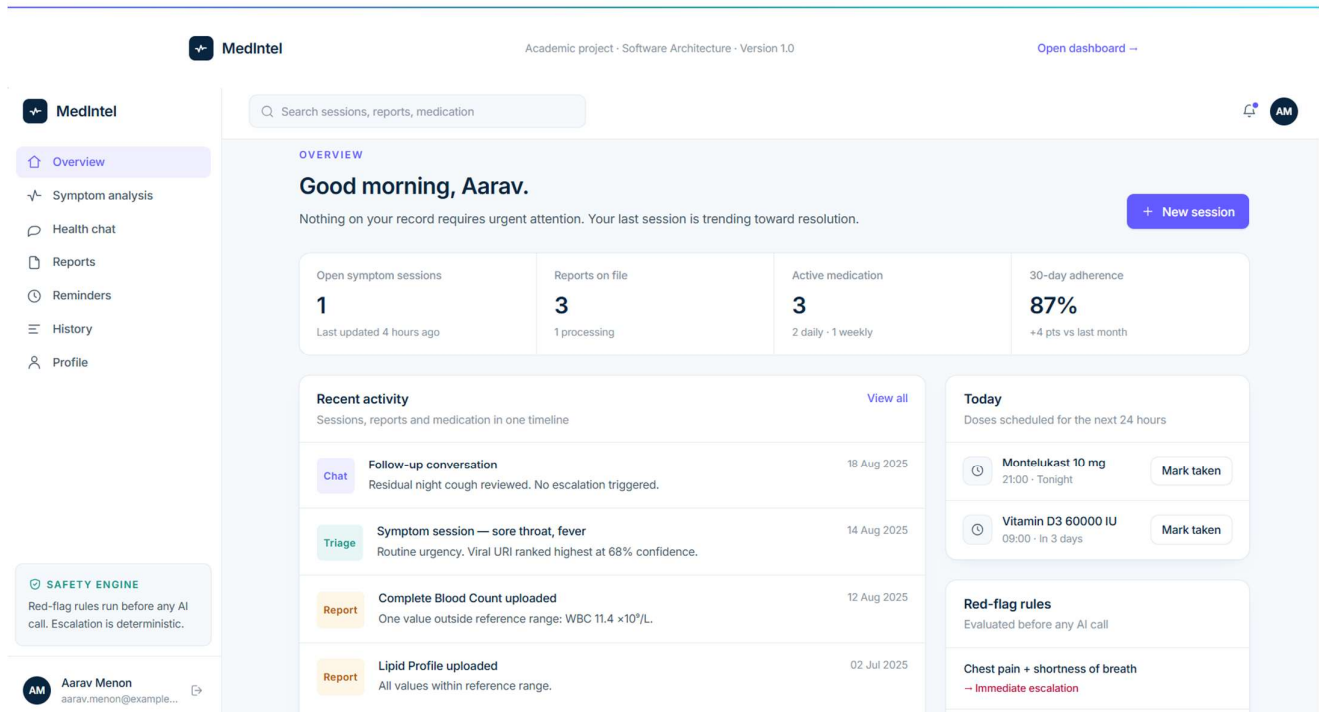
THE GUARDRAIL PRINCIPLE

A probabilistic component never sits alone on a safety-critical path.

Every AI response is schema-validated, checked against banned content, and wrapped with a confidence band, an urgency level and a mandatory notice to consult a physician. Red-flag symptom combinations bypass the model entirely.

MEDINTEL IS NOT

- A diagnosis — output is possibility, never verdict
- A prescription engine — no drug names, no dosages
- An emergency service — it escalates, it does not treat
- A replacement for a licensed clinician
- A store of record for legal or insurance purposes



Purpose: Acts as the primary central command hub displaying the user's current health status and recent events at a glance.

Main Components: KPI summary cards (Open symptom sessions, Reports on file, Active medication, 30-day adherence), *Recent activity* chronological feed, *Today* medication queue, and a persistent *Red-flag rules* indicator.

Buttons/Actions: + New session, View all, Mark taken (per scheduled dose), and global search.

Functionality: Aggregates health metrics, allows quick logging of daily doses, and gives immediate access to ongoing triage workflows.

MedIntel

Overview

Symptom analysis

Health chat

Reports

Reminders

History

Profile

SAFETY ENGINE

Red-flag rules run before any AI call. Escalation is deterministic.

Aarav Menon
aarav.menon@example...

Search sessions, reports, medication

MODULE 02

Symptom analysis

Structured intake runs through the deterministic rules engine before the AI is called. The result is a ranked list of possibilities — never a diagnosis.

1 Symptoms

2 Details

3 Result

What are you experiencing?

Select everything that applies — you can refine next.

Sore throat

Fever

Cough

Headache

Fatigue

Shortness of breath

Chest pain

Nausea

Dizziness

Body ache

Runny nose

Loss of appetite

Anything not listed above

Describe in your own words...

Free text is normalised and redacted before it leaves the API layer.

BackContinue →

Session summary

Symptoms

3 selected

Severity

4 / 10

Red flags

0 matched

Confidence

72%

Pipeline trace

What the request passed through

1 Normalise

2 Redact PII

Purpose: Guides users through structured clinical symptom intake before AI triage runs.

Main Components: Multi-step progress bar (*Symptoms* → *Details* → *Result*), multi-select symptom tags (Sore throat, Fever, Cough, etc.), free-text custom description input, *Session summary* widget, and a *Pipeline trace* view.

Buttons/Actions: Tag selection toggles, Back, Continue →.

Functionality: Normalizes selected symptoms, redacts PII, and runs deterministic red-flag triage rules to compute an urgency score and possibility ranking.

MedIntel

Overview

Symptom analysis

Health chat

Reports

Reminders

History

Profile

SAFETY ENGINE

Red-flag rules run before any AI call. Escalation is deterministic.

Aarav Menon
aarav.menon@example...

Search sessions, reports, medication

MODULE 03

Health chat

Follow-up conversation that carries your session context and stored history into every turn — under the same guardrails as triage.

Session #4821

Sore throat, fever · opened 14 Aug

Routine

Good morning, Aarav. I have your session from 14 Aug on file — sore throat, mild fever, day three. How are you feeling today?

Fever is gone but the cough is still there, mostly at night.

A residual night cough after a viral infection is common and can persist for one to three weeks. Since your fever has resolved, the trend is in the right direction.

Two things worth watching: any wheeze given your asthma history, and whether the cough starts producing coloured sputum.

Should I be worried about the night cough?How does my asthma change this?

When should I see a doctor in person?

Ask a follow-up questionSend

Context in this conversation

What the model was given

PROFILE

Age 27 · male · mild asthma

ALLERGIES

Penicillin, dust mites

LAST SESSION

14 Aug · viral URI, 68%

RECENT REPORT

CBC · 1 value out of range

Applied guardrails

✓ PII redacted before the provider boundary

✓ Response schema-validated on return

Purpose: Enables contextual, multi-turn follow-up conversations based on previous triage sessions and uploaded reports.

Main Components: Conversational chat stream with AI and user message bubbles, suggested follow-up query chips, contextual sidebar (active profile, allergies, last session, lab reports), and applied guardrail checklist.

Buttons/Actions: Quick suggestion chips, message input bar, Send button.

Functionality: Provides AI-driven medical explanations that inherit user context without exposing raw PII to the language model.

MedIntel

Search sessions, reports, medication

Overview
Symptom analysis
Health chat
Reports
Reminders
History
Profile

Medical reports

Upload a PDF or image. Extraction runs off the request path on a worker queue, then attaches a plain-language summary to your timeline.

Upload report

Drop a report here
PDF, JPG or PNG - up to 10 MB
Browse files

On file
3 reports

- Complete Blood Count**
Apollo Diagnostics - 12 Aug 2025 Summarised
- Lipid Profile**
Metropolis - 02 Jul 2025 Summarised

Complete Blood Count
Apollo Diagnostics - 12 Aug 2025 1 value flagged

EXTRACTED VALUES

MARKER	RESULT	REFERENCE
Haemoglobin	14.2 g/dL	13.0 – 17.0
WBC count	11.4 ×10 ⁹ /L HIGH	4.0 – 11.0
Platelets	265 ×10 ⁹ /L	150 – 410
ESR	18 mm/hr	0 – 22

SUMMARY

Most markers sit comfortably within the reference range. The white cell count is mildly elevated, which is a common finding during or shortly after an infection and is consistent with the symptom session recorded on 14 August. It is worth repeating once the current illness has resolved so the trend can be compared.

SAFETY ENGINE
Red-flag rules run before any AI call. Escalation is deterministic.

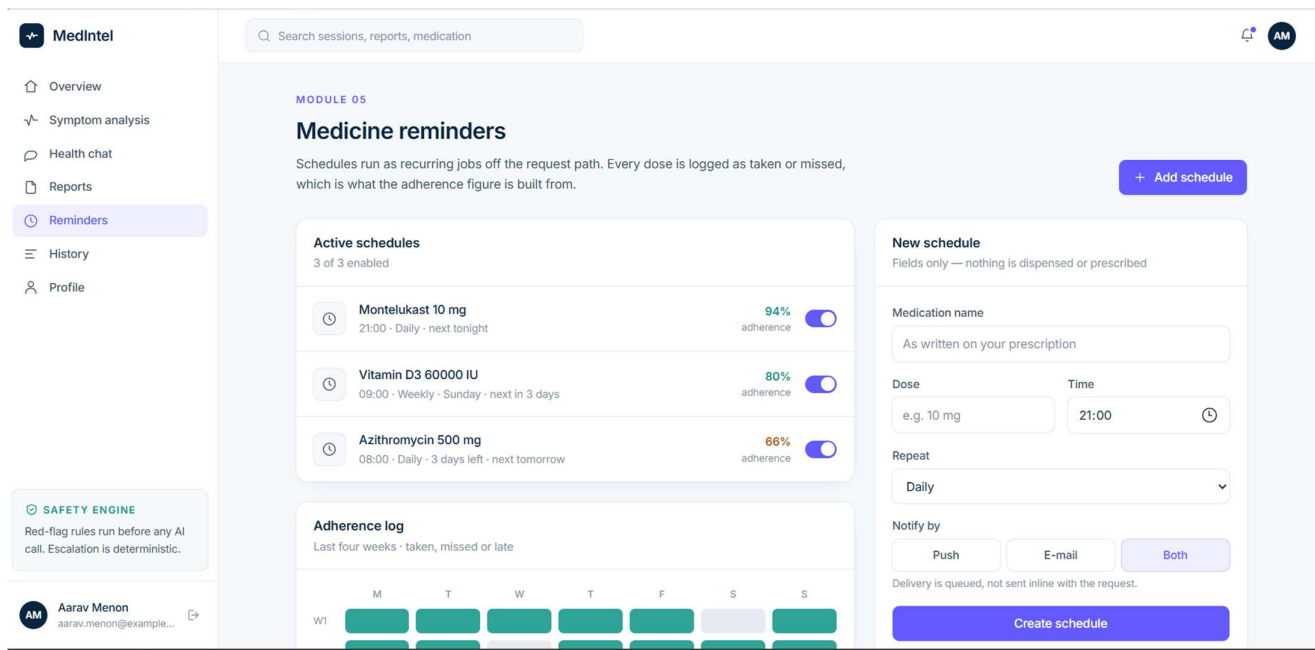
Aarav Menon
aarav.menon@example...

Purpose: Uploads, processes, and extracts plain-language insights from diagnostic lab documents.

Main Components: Drag-and-drop file upload zone, *On file* document listing, parsed *Extracted Values* table (markers, results, reference ranges, flag status), and an automated summary panel.

Buttons/Actions: Browse files, Upload report.

Functionality: Runs OCR asynchronously on uploaded PDFs/images, flags abnormal values (e.g., high WBC count), and generates plain-language clinical interpretations.

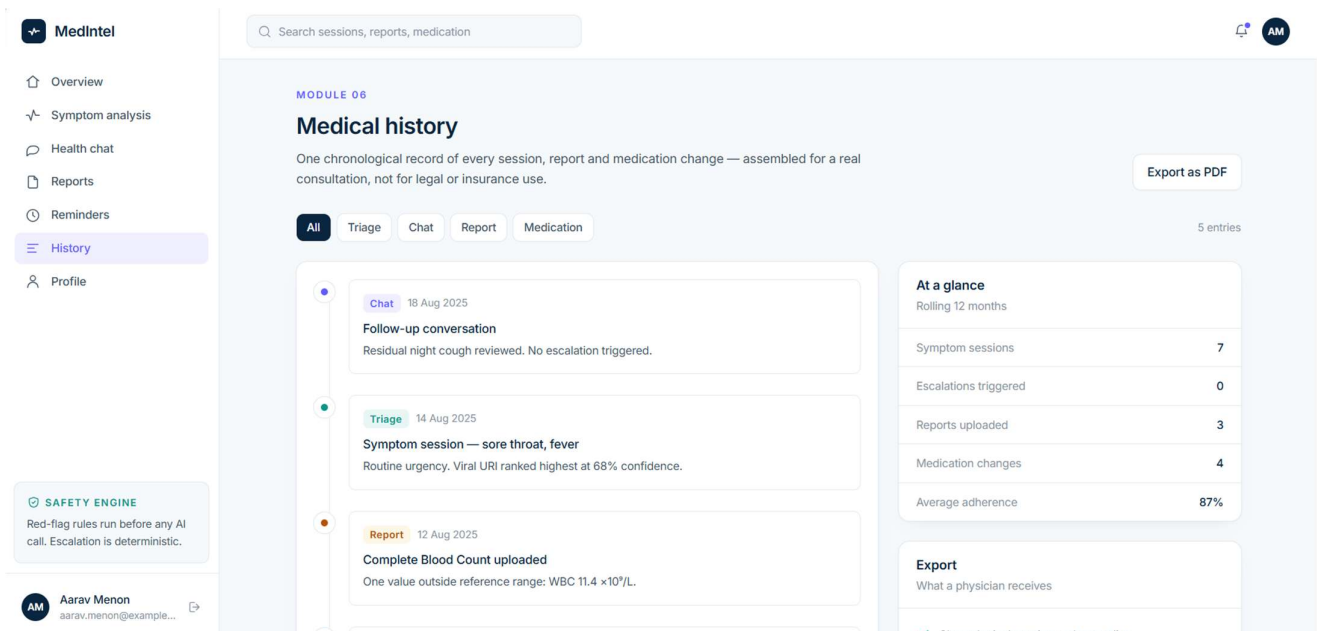


Purpose: Manages recurring medication schedules and monitors adherence over time.

Main Components: *Active schedules* list with adherence percentages, toggle switches, 4-week adherence heatmap grid, and a *New schedule* builder form (Medication name, Dose, Time, Repeat frequency, Notification channel).

Buttons/Actions: Schedule toggle switches, + Add schedule, Push / E-mail / Both selectors, Create schedule.

Functionality: Tracks medication compliance, triggers background reminders off the main request thread, and calculates rolling adherence rates.



Purpose: Serves as a single chronological timeline of all consultations, reports, and medication changes.

Main Components: Filter pills (All, Triage, Chat, Report, Medication), timeline entry stream with color-coded badges, *At a glance* 12-month rolling summary card, and an *Export* feature breakdown.

Buttons/Actions: Filter toggles, Export as PDF.

Functionality: Compiles structured patient history into a downloadable, physician-ready report for real-world consultations.

MedIntel

Search sessions, reports, medication

Profile & account

Everything here is injected as context before the AI is called, and every field is redacted at the provider boundary.

Personal details

Aarav Menon
aarav.menon@example.com [Change photo](#)

Full name: Aarav Menon Email: aarav.menon@example.com

Age: 27 Sex: Male

Clinical background
Raises or lowers baseline probability during triage

Allergies
Penicillin Dust mites [+ Add](#)

Chronic conditions
Asthma [+ Add](#)

Security

Password
Updated 3 months ago [Change](#)

Two-factor auth
Not enabled [Enable](#)

Active sessions
2 devices [Review](#)

Data & privacy

- ☒ Identifiers stripped before any AI request
- ☒ Encrypted in transit and at rest
- ☒ Per-resource ownership checks on every read
- ☒ Append-only audit log of access

[Delete account and all records](#)

[Save changes](#)

SAFETY ENGINE
Red-flag rules run before any AI call. Escalation is deterministic.

Aarav Menon
aarav.menon@example.com

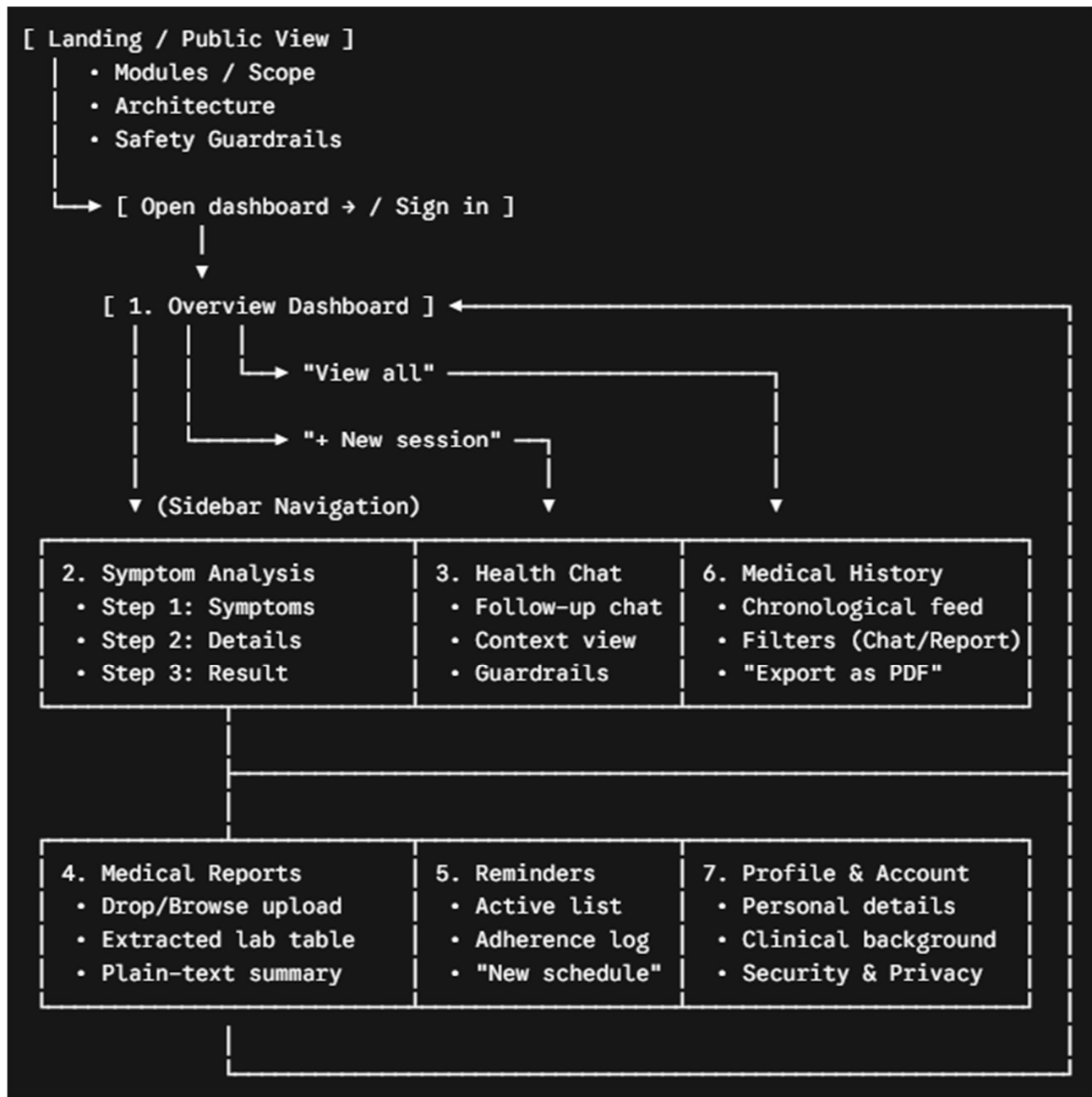
Purpose: Manages user demographics, clinical baseline history, authentication security, and data privacy settings.

Main Components: Personal details form (Name, Email, Age, Sex), *Clinical background* tag selectors (Allergies, Chronic conditions), *Security* card (Password, 2FA, Active sessions), and *Data & privacy* audit checklist.

Buttons/Actions: Save changes, Change photo, + Add tags, Change (Password), Enable (2FA), Review (Sessions), Delete account and all records.

Functionality: Updates the patient baseline context that is injected into all AI reasoning steps, manages account security, and enforces privacy controls.

navigation flow between the major screens.



Code implementation

1. Entry & Config

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet" />
    <title>MedIntel</title>
    <script>
      // Surfaces startup failures on screen instead of Leaving a blank page.
      (function () {
        function show(title, detail) {
          var root = document.getElementById('root')
          if (!root) return
          root.innerHTML =
            '<div style="font:14px/1.6 ui-monospace,Consolas,monospace;color:#0a2540;padding:32px;max-width:900px">' +
            '<p style="color:#be123c;font-weight:700;margin:0 0 8px">' + title + '</p>' +
            '<pre style="white-space:pre-wrap;background:#f6f9fc;border:1px solid #e6ebf1;border-radius:8px;padding:16px;margin:0">' +
            String(detail).replace(/</g, '&lt;') + '</pre></div>'
        }
        window.addEventListener('error', function (e) {
          show('Script error', (e.error && e.error.stack) || e.message || e.filename)
        })
        window.addEventListener('unhandledrejection', function (e) {
          show('Unhandled promise rejection', (e.reason && e.reason.stack) || e.reason)
        })
      })()
    </script>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
```

```
</html>
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import ErrorBoundary from './components/ErrorBoundary.jsx'
import './index.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <ErrorBoundary>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </ErrorBoundary>
  </StrictMode>,
)
```

src/App.jsx

```
import { Routes, Route, Navigate, useLocation } from 'react-router-dom'
import { useEffect } from 'react'
import AppLayout from './components/AppLayout'
import Landing from './pages/Landing'
import SignIn from './pages/SignIn'
import Dashboard from './pages/Dashboard'
import Symptoms from './pages/Symptoms'
import Chat from './pages/Chat'
import Reports from './pages/Reports'
import Reminders from './pages/Reminders'
import History from './pages/History'
import Profile from './pages/Profile'

function ScrollToTop() {
  const { pathname } = useLocation()
  useEffect(() => {
    window.scrollTo(0, 0)
  }, [pathname])
  return null
}
```

```

export default function App() {
  return (
    <>
      <ScrollToTop />
      <Routes>
        <Route path="/" element={<Landing />} />
        <Route path="/signin" element={<SignIn />} />
        <Route path="/app" element={<AppLayout />}>
          <Route index element={<Dashboard />} />
          <Route path="symptoms" element={<Symptoms />} />
          <Route path="chat" element={<Chat />} />
          <Route path="reports" element={<Reports />} />
          <Route path="reminders" element={<Reminders />} />
          <Route path="history" element={<History />} />
          <Route path="profile" element={<Profile />} />
        </Route>
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </>
  )
}

```

src/index.css

```

@import "tailwindcss";

@theme {
  --font-sans: "Inter", ui-sans-serif, system-ui, -apple-system, "Segoe UI", sans-serif;

  --color-ink: #0a2540;
  --color-slate: #425466;
  --color-muted: #8792a2;
  --color-line: #e6ebf1;
  --color-surface: #f6f9fc;
  --color-brand: #635bff;
  --color-brand-dark: #4e46e5;
  --color-brand-soft: #f0efff;
  --color-teal: #0d9488;
  --color-teal-soft: #e8f7f5;
  --color-amber: #b45309;
  --color-amber-soft: #fef6e7;
  --color-rose: #be123c;
  --color-rose-soft: #fdf0f3;

  --shadow-card: 0 1px 2px rgba(10, 37, 64, .05), 0 8px 24px -12px rgba(10, 37, 64, .12);
}

```

```

--shadow-lift: 0 2px 4px rgba(10, 37, 64, .06), 0 18px 40px -16px rgba(10, 37, 64, .22);
}

html { -webkit-font-smoothing: antialiased; }

body {
  margin: 0;
  font-family: var(--font-sans);
  color: var(--color-ink);
  background: #fff;
  letter-spacing: -0.011em;
}

::selection { background: #ddd9ff; }

.gradient-rule {
  background: linear-gradient(90deg, #635bff 0%, #0d9488 50%, #00d4ff 100%);
}

.hero-wash {
  background:
    radial-gradient(900px 420px at 88% -10%, rgba(99, 91, 255, .16), transparent 60%),
    radial-gradient(700px 380px at 5% 0%, rgba(13, 148, 136, .12), transparent 60%);
}

.grid-lines {
  background-image:
    linear-gradient(to right, rgba(10, 37, 64, .045) 1px, transparent 1px),
    linear-gradient(to bottom, rgba(10, 37, 64, .045) 1px, transparent 1px);
  background-size: 56px 56px;
}

@keyframes rise {
  from { opacity: 0; transform: translateY(8px); }
  to { opacity: 1; transform: none; }
}

.rise { animation: rise .5s cubic-bezier(.22,.61,.36,1) both; }

*:focus-visible {
  outline: 2px solid var(--color-brand);
  outline-offset: 2px;
  border-radius: 6px;
}

```

2. Data Layer

src/data/mock.js

```
export const user = {
  name: 'Aarav Menon',
  email: 'aarav.menon@example.com',
  initials: 'AM',
  age: 27,
  sex: 'Male',
  allergies: ['Penicillin', 'Dust mites'],
  conditions: ['Mild asthma'],
}

export const nav = [
  { to: '/app', label: 'Overview', icon: 'home', end: true },
  { to: '/app/symptoms', label: 'Symptom analysis', icon: 'pulse' },
  { to: '/app/chat', label: 'Health chat', icon: 'chat' },
  { to: '/app/reports', label: 'Reports', icon: 'file' },
  { to: '/app/reminders', label: 'Reminders', icon: 'clock' },
  { to: '/app/history', label: 'History', icon: 'timeline' },
  { to: '/app/profile', label: 'Profile', icon: 'user' },
]

export const triageResult = {
  urgency: 'Routine',
  tone: 'teal',
  confidence: 0.72,
  redFlags: [],
  conditions: [
    { name: 'Viral upper respiratory infection', likelihood: 0.68, note: 'Consistent with onset, fever pattern and absence of localised pain.' },
    { name: 'Acute bacterial sinusitis', likelihood: 0.21, note: 'Consider if symptoms persist beyond 10 days or worsen after improvement.' },
    { name: 'Seasonal allergic rhinitis', likelihood: 0.11, note: 'History of dust-mite allergy raises baseline probability.' },
  ],
  advice: [
    'Rest and maintain fluid intake for the next 48 hours.',
    'Monitor temperature twice daily and record it in your history.',
    'Seek in-person care if breathing becomes difficult or fever exceeds 39 °C.',
  ],
}

export const redFlagRules = [
  { rule: 'Chest pain + shortness of breath', action: 'Immediate escalation', tone: 'rose' },
  { rule: 'Sudden unilateral weakness', action: 'Immediate escalation', tone: 'rose' },
]
```

```
{ rule: 'Fever > 39 °C for 3+ days', action: 'Urgent review', tone: 'amber' },
]

export const messages = [
  { from: 'bot', text: 'Good morning, Aarav. I have your session from 14 Aug on file – sore throat, mild fever, day three. How are you feeling today?' },
  { from: 'user', text: 'Fever is gone but the cough is still there, mostly at night.' },
  { from: 'bot', text: 'A residual night cough after a viral infection is common and can persist for one to three weeks. Since your fever has resolved, the trend is in the right direction.\n\nTwo things worth watching: any wheeze given your asthma history, and whether the cough starts producing coloured sputum.' },
  { from: 'bot', text: 'This is decision support, not a diagnosis. If the cough wakes you repeatedly or you feel breathless, please see a physician.', meta: true },
]

export const reports = [
  { name: 'Complete Blood Count', lab: 'Apollo Diagnostics', date: '12 Aug 2025', status: 'Summarised', tone: 'teal', flags: 1 },
  { name: 'Lipid Profile', lab: 'Metropolis', date: '02 Jul 2025', status: 'Summarised', tone: 'teal', flags: 0 },
  { name: 'Chest X-Ray', lab: 'City Imaging', date: '28 Jun 2025', status: 'Processing', tone: 'amber', flags: 0 },
]

export const reminders = [
  { drug: 'Montelukast 10 mg', time: '21:00', freq: 'Daily', next: 'Tonight', adherence: 0.94, tone: 'teal' },
  { drug: 'Vitamin D3 60000 IU', time: '09:00', freq: 'Weekly · Sunday', next: 'In 3 days', adherence: 0.8, tone: 'teal' },
  { drug: 'Azithromycin 500 mg', time: '08:00', freq: 'Daily · 3 days left', next: 'Tomorrow', adherence: 0.66, tone: 'amber' },
]

export const timeline = [
  { date: '18 Aug 2025', kind: 'Chat', tone: 'brand', title: 'Follow-up conversation', body: 'Residual night cough reviewed. No escalation triggered.' },
  { date: '14 Aug 2025', kind: 'Triage', tone: 'teal', title: 'Symptom session – sore throat, fever', body: 'Routine urgency. Viral URI ranked highest at 68% confidence.' },
  { date: '12 Aug 2025', kind: 'Report', tone: 'amber', title: 'Complete Blood Count uploaded', body: 'One value outside reference range: WBC  $11.4 \times 10^9/L$ .' },
  { date: '02 Jul 2025', kind: 'Report', tone: 'amber', title: 'Lipid Profile uploaded', body: 'All values within reference range.' },
  { date: '28 Jun 2025', kind: 'Medication', tone: 'brand', title: 'Montelukast schedule created', body: 'Daily 21:00, ongoing. Adherence tracked from this date.' },
]
```


3. Shared Components

`src/components/ui.jsx`

```
import { Link } from 'react-router-dom'

export function Button({ as = 'button', variant = 'primary', size = 'md', className = '',
...props }) {
  const base = 'inline-flex items-center justify-center gap-2 rounded-lg font-medium
transition-all duration-150 disabled:opacity-50 disabled:pointer-events-none whitespace-
nowrap'
  const sizes = {
    sm: 'h-8 px-3 text-[13px]',
    md: 'h-10 px-4 text-sm',
    lg: 'h-11 px-5 text-[15px]',
  }
  const variants = {
    primary: 'bg-brand text-white shadow-[0_1px_2px_rgba(10,37,64,.16)] hover:bg-brand-
dark hover:-translate-y-px active:translate-y-0',
    dark: 'bg-ink text-white hover:bg-[#123456] hover:-translate-y-px',
    secondary: 'bg-white text-ink ring-1 ring-line hover:ring-[#cfd8e3] shadow-
[0_1px_2px_rgba(10,37,64,.06)]',
    ghost: 'text-slate hover:text-ink hover:bg-surface',
    soft: 'bg-brand-soft text-brand hover:bg-[#e6e4ff]',
  }
  const Cmp = as === 'link' ? Link : as
  return <Cmp className={` ${base} ${sizes[size]} ${variants[variant]} ${className}`}
{...props} />
}

export function Card({ className = '', children, ...props }) {
  return (
    <div className={`rounded-xl bg-white ring-1 ring-line shadow-card ${className}`}
{...props}>
      {children}
    </div>
  )
}

export function CardHead({ title, sub, action }) {
  return (
    <div className="flex items-start justify-between gap-4 border-b border-line px-5 py-
4">
      <div>
        <h3 className="text-[15px] font-semibold text-ink">{title}</h3>
        {sub && <p className="mt-0.5 text-[13px] text-muted">{sub}</p>}
```

```

    </div>
    {action}
  </div>
)
}

const tones = {
  brand: 'bg-brand-soft text-brand',
  teal: 'bg-teal-soft text-teal',
  amber: 'bg-amber-soft text-amber',
  rose: 'bg-rose-soft text-rose',
  slate: 'bg-surface text-slate',
}

export function Badge({ tone = 'slate', className = '', children }) {
  return (
    <span className={`inline-flex items-center gap-1.5 rounded-md px-2 py-0.5 text-[11.5px] font-semibold tracking-wide ${tones[tone]} ${className}`}>
      {children}
    </span>
  )
}

export function Dot({ tone = 'slate' }) {
  const c = { brand: 'bg-brand', teal: 'bg-teal', amber: 'bg-amber', rose: 'bg-rose', slate: 'bg-muted' }[tone]
  return <span className={`size-1.5 rounded-full ${c}`} />
}

export function Field({ label, hint, children }) {
  return (
    <label className="block">
      <span className="mb-1.5 block text-[13px] font-medium text-slate">{label}</span>
      {children}
      {hint && <span className="mt-1.5 block text-[12px] text-muted">{hint}</span>}
    </label>
  )
}

export const inputCls =
  'w-full rounded-lg bg-white px-3 py-2.5 text-sm text-ink ring-1 ring-line shadow-[0_1px_2px_rgba(10,37,64,.05)] placeholder:text-muted transition focus:outline-none focus:ring-2 focus:ring-brand'

export function PageHead({ eyebrow, title, sub, children }) {
  return (
    <div className="mb-7 flex flex-wrap items-end justify-between gap-4">

```

```

    <div>
      {eyebrow && (
        <p className="mb-2 text-[11.5px] font-semibold uppercase tracking-[0.14em] text-brand">{eyebrow}</p>
      )}
      <h1 className="text-[26px] font-semibold tracking-[-0.02em] text-ink">{title}</h1>
      {sub && <p className="mt-1.5 max-w-2xl text-[14.5px] leading-relaxed text-slate">{sub}</p>}
    </div>
    {children}
  </div>
)
}

export function Empty({ title, sub, action }) {
  return (
    <div className="flex flex-col items-center justify-center px-6 py-16 text-center">
      <div className="mb-3 size-9 rounded-lg bg-surface ring-1 ring-line" />
      <p className="text-sm font-medium text-ink">{title}</p>
      <p className="mt-1 max-w-sm text-[13px] text-muted">{sub}</p>
      {action && <div className="mt-4">{action}</div>}
    </div>
  )
}

```

src/components/AppLayout.jsx

```

import { useState } from 'react'
import { NavLink, Outlet, Link } from 'react-router-dom'
import Icon from '../Icon'
import Logo from '../Logo'
import { nav, user } from '../data/mock'

export default function AppLayout() {
  const [open, setOpen] = useState(false)

  return (
    <div className="min-h-screen bg-surface">
      <aside
        className={`fixed inset-y-0 left-0 z-40 flex w-[248px] flex-col border-r border-line bg-white transition-transform duration-200 lg:translate-x-0 ${
          open ? 'translate-x-0' : '-translate-x-full'
        }`}
      >

```

```

<div className="flex h-16 items-center px-5">
  <Link to="/"><Logo /></Link>
</div>

<nav className="flex-1 space-y-0.5 px-3 py-2">
  {nav.map((item) => (
    <NavLink
      key={item.to}
      to={item.to}
      end={item.end}
      onClick={() => setOpen(false)}
      className={({ isActive }) =>
        `flex items-center gap-3 rounded-lg px-3 py-2 text-[13.5px] font-medium
transition-colors ${
          isActive ? 'bg-brand-soft text-brand' : 'text-slate hover:bg-surface
hover:text-ink'
        }`
      }
    >
      <Icon name={item.icon} className="size-[18px]" />
      {item.label}
    </NavLink>
  ))}
</nav>

<div className="m-3 rounded-lg bg-surface p-3.5 ring-1 ring-line">
  <div className="mb-1.5 flex items-center gap-1.5 text-teal">
    <Icon name="shield" className="size-[15px]" strokeWidth={1.8} />
    <span className="text-[11.5px] font-semibold uppercase tracking-
[0.1em]">Safety engine</span>
  </div>
  <p className="text-[12.5px] leading-relaxed text-slate">
    Red-flag rules run before any AI call. Escalation is deterministic.
  </p>
</div>

<div className="border-t border-line p-3">
  <Link to="/signin" className="flex items-center gap-3 rounded-lg px-2 py-2
hover:bg-surface">
    <span className="grid size-8 place-items-center rounded-full bg-ink text-
[11.5px] font-semibold text-white">
      {user.initials}
    </span>
    <span className="min-w-0 flex-1">
      <span className="block truncate text-[13px] font-medium text-
ink">{user.name}</span>
    </span>
  </Link>

```

```

        <span className="block truncate text-[11.5px] text-
muted">{user.email}</span>
      </span>
      <Icon name="logout" className="size-4 text-muted" />
    </Link>
  </div>
</aside>

{open && <div className="fixed inset-0 z-30 bg-ink/20 lg:hidden" onClick={() =>
setOpen(false)} />}

<div className="lg:pl-[248px]">
  <header className="sticky top-0 z-20 flex h-16 items-center gap-3 border-b border-
line bg-white/85 px-5 backdrop-blur-md sm:px-8">
    <button
      onClick={() => setOpen(true)}
      className="-ml-1 grid size-9 place-items-center rounded-lg text-slate
hover:bg-surface lg:hidden"
      aria-label="Open navigation"
    >
      <Icon name="timeline" />
    </button>

    <div className="relative hidden max-w-sm flex-1 sm:block">
      <Icon name="search" className="pointer-events-none absolute left-3 top-1/2
size-4 -translate-y-1/2 text-muted" />
      <input
        placeholder="Search sessions, reports, medication"
        className="h-9 w-full rounded-lg bg-surface pl-9 pr-3 text-[13px] text-ink
ring-1 ring-line transition placeholder:text-muted focus:outline-none focus:ring-2
focus:ring-brand"
      />
    </div>

    <div className="ml-auto flex items-center gap-1.5">
      <button className="relative grid size-9 place-items-center rounded-lg text-
slate hover:bg-surface" aria-label="Notifications">
        <Icon name="bell" className="size-[18px]" />
        <span className="absolute right-2 top-2 size-1.5 rounded-full bg-brand ring-
2 ring-white" />
      </button>
      <span className="grid size-8 place-items-center rounded-full bg-ink text-
[11.5px] font-semibold text-white">
        {user.initials}
      </span>
    </div>
  </header>

```

```

    <main className="mx-auto max-w-6xl px-5 py-8 sm:px-8">
      <Outlet />
    </main>
  </div>
</div>
)
}

```

src/components/Icon.jsx

```

const paths = {
  pulse: 'M2 12h4l3 8 4-16 3 8h6',
  chat: 'M21 12a8 8 0 0 1-8 8H7l-4 3v-7a8 8 0 0 1 8-8h2a8 8 0 0 1 8 4Z',
  file: 'M14 3v5h5M14 3H7a2 2 0 0 0-2 2v14a2 2 0 0 0 2 2h10a2 2 0 0 0 2-2V8l-5-5Z',
  clock: 'M12 21a9 9 0 1 0 0-18 9 9 0 0 0 0 18ZM12 7v5l3 2',
  timeline: 'M4 6h16M4 12h10M4 18h13',
  user: 'M20 21a8 8 0 1 0-16 0M12 11a4 4 0 1 0 0-8 4 4 0 0 0 0 8Z',
  home: 'M3 10.5 12 3l9 7.5M6 9v11h12V9',
  shield: 'M12 3l8 3v6c0 5-3.4 8-8 9-4.6-1-8-4-8-9V6l8-3ZM9 12l2 2 4-4',
  arrow: 'M5 12h13M13 6l6 6-6 6',
  plus: 'M12 5v14M5 12h14',
  check: 'M4 12.5 9 17.5 20 6.5',
  alert: 'M12 8v5M12 16.5v.5M10.3 3.9 2.7 17a2 2 0 0 0 1.7 3h15.2a2 2 0 0 0 1.7-3L13.7 3.9a2 2 0 0 0-3.4 0Z',
  send: 'M4 12 20 4l-7 16-2-7-7-1Z',
  search: 'M11 19a8 8 0 1 0 0-16 8 8 0 0 0 0 16ZM21 21l-4.3-4.3',
  bell: 'M18 9a6 6 0 1 0-12 0c0 6-2 7-2 7h16s-2-1-2-7M10.5 20a2 2 0 0 0 3 0',
  logout: 'M15 17l5-5-5-5M20 12H9M12 4H6a2 2 0 0 0-2 2v12a2 2 0 0 0 2 2h6',
  upload: 'M12 16V4M7 9l5-5 5 5M4 17v2a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2v-2',
  spark: 'M12 3l1.9 5.6L19.5 10l-5.6 1.9L12 17.5l-1.9-5.6L4.5 10l5.6-1.4L12 3Z',
}

export default function Icon({ name, className = 'size-5', strokeWidth = 1.6 }) {
  return (
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth={strokeWidth}
      strokeLinecap="round" strokeLinejoin="round" className={className} aria-
      hidden="true">
      <path d={paths[name]} />
    </svg>
  )
}

```

4. Page Views

src/pages/Dashboard.jsx

```
import { Link } from 'react-router-dom'
import Icon from '../components/Icon'
import { Card, CardHead, Badge, Button, PageHead } from '../components/ui'
import { user, timeline, reminders, redFlagRules } from '../data/mock'

const stats = [
  { label: 'Open symptom sessions', value: '1', delta: 'Last updated 4 hours ago' },
  { label: 'Reports on file', value: '3', delta: '1 processing' },
  { label: 'Active medication', value: '3', delta: '2 daily · 1 weekly' },
  { label: '30-day adherence', value: '87%', delta: '+4 pts vs last month' },
]

const shortcuts = [
  { to: '/app/symptoms', icon: 'pulse', title: 'Start a symptom session', body: 'Structured intake, rules first, AI second.' },
  { to: '/app/chat', icon: 'chat', title: 'Ask a follow-up', body: 'Context-aware chat over your history.' },
  { to: '/app/reports', icon: 'upload', title: 'Upload a report', body: 'PDF or image – OCR runs in the background.' },
]

export default function Dashboard() {
  return (
    <>
      <PageHead
        eyebrow="Overview"
        title={`Good morning, ${user.name.split(' ')[0]}.`}
        sub="Nothing on your record requires urgent attention. Your last session is trending toward resolution."
      >
        <Button as="link" to="/app/symptoms">
          <Icon name="plus" className="size-4" /> New session
        </Button>
      </PageHead>

      <div className="grid gap-px overflow-hidden rounded-xl bg-line ring-1 ring-line sm:grid-cols-2 lg:grid-cols-4">
        {stats.map((s) => (
          <div key={s.label} className="bg-white px-5 py-4">
            <p className="text-[12.5px] font-medium text-muted">{s.label}</p>
            <p className="mt-1.5 text-[26px] font-semibold tracking-[-0.03em] text-ink">{s.value}</p>
          </div>
        ))}
      </div>
    </>
  )
}
```

```

        <p className="mt-0.5 text-[12px] text-muted">{s.delta}</p>
      </div>
    )))
  </div>

  <div className="mt-6 grid gap-6 lg:grid-cols-3">
    <Card className="lg:col-span-2">
      <CardHead
        title="Recent activity"
        sub="Sessions, reports and medication in one timeline"
        action={<Link to="/app/history" className="text-[13px] font-medium text-brand
hover:text-brand-dark">View all</Link>}
      />
      <ul className="divide-y divide-line">
        {timeline.slice(0, 4).map((t) => (
          <li key={t.title} className="flex gap-4 px-5 py-4 transition-colors
hover:bg-surface">
            <Badge tone={t.tone} className="mt-0.5 shrink-0">{t.kind}</Badge>
            <div className="min-w-0 flex-1">
              <p className="text-[13.5px] font-medium text-ink">{t.title}</p>
              <p className="mt-0.5 text-[13px] leading-relaxed text-
slate">{t.body}</p>
            </div>
            <span className="shrink-0 text-[12px] text-muted">{t.date}</span>
          </li>
        )))
      </ul>
    </Card>

    <div className="space-y-6">
      <Card>
        <CardHead title="Today" sub="Doses scheduled for the next 24 hours" />
        <ul className="divide-y divide-line">
          {reminders.slice(0, 2).map((r) => (
            <li key={r.drug} className="flex items-center gap-3 px-5 py-3.5">
              <span className="grid size-8 shrink-0 place-items-center rounded-lg bg-
surface text-slate ring-1 ring-line">
                <Icon name="clock" className="size-4" />
              </span>
              <div className="min-w-0 flex-1">
                <p className="truncate text-[13.5px] font-medium text-
ink">{r.drug}</p>
                <p className="text-[12px] text-muted">{r.time} · {r.next}</p>
              </div>
              <Button size="sm" variant="secondary">Mark taken</Button>
            </li>
          )))
        </ul>
      </Card>
    </div>
  </div>

```



```

        </ul>
      </Card>

      <Card>
        <CardHead title="Red-flag rules" sub="Evaluated before any AI call" />
        <ul className="divide-y divide-line">
          {redFlagRules.map((r) => (
            <li key={r.rule} className="px-5 py-3.5">
              <p className="text-[13px] font-medium text-ink">{r.rule}</p>
              <p className={`mt-1 text-[12px] font-medium ${r.tone === 'rose' ? 'text-rose' : 'text-amber'} `}>
                → {r.action}
              </p>
            </li>
          )))}
        </ul>
      </Card>
    </div>
  </div>

  <div className="mt-6 grid gap-py overflow-hidden rounded-xl bg-line ring-1 ring-line
sm:grid-cols-3">
    {shortcuts.map((s) => (
      <Link key={s.to} to={s.to} className="group bg-white px-5 py-5 transition-colors
hover:bg-surface">
        <span className="grid size-8 place-items-center rounded-lg bg-brand-soft text-
brand">
          <Icon name={s.icon} className="size-[18px]" />
        </span>
        <p className="mt-3.5 flex items-center gap-1.5 text-[14px] font-semibold text-
ink">
          {s.title}
          <Icon name="arrow" className="size-3.5 text-muted transition-transform
group-hover:translate-x-0.5" />
        </p>
        <p className="mt-1 text-[13px] text-slate">{s.body}</p>
      </Link>
    )))}
  </div>
</>
)
}

```

src/pages/Symptoms.jsx

```

import { useState } from 'react'
import Icon from '../components/Icon'
import { Card, CardHead, Badge, Button, Field, inputCls, PageHead } from
'../components/ui'
import { triageResult } from '../data/mock'

const symptomOptions = [
  'Sore throat', 'Fever', 'Cough', 'Headache', 'Fatigue', 'Shortness of breath',
  'Chest pain', 'Nausea', 'Dizziness', 'Body ache', 'Runny nose', 'Loss of appetite',
]

const steps = ['Symptoms', 'Details', 'Result']

function Meter({ value, tone = 'brand' }) {
  const bar = { brand: 'bg-brand', teal: 'bg-teal', amber: 'bg-amber', rose: 'bg-rose'
}[tone]
  return (
    <div className="h-1.5 w-full overflow-hidden rounded-full bg-line">
      <div className={`h-full rounded-full ${bar} transition-[width] duration-500`}
style={{ width: `${value * 100}%` }} />
    </div>
  )
}

export default function Symptoms() {
  const [step, setStep] = useState(0)
  const [selected, setSelected] = useState(['Sore throat', 'Fever', 'Cough'])
  const [severity, setSeverity] = useState(4)

  const toggle = (s) =>
    setSelected((prev) => (prev.includes(s) ? prev.filter((x) => x !== s) : [...prev, s]))

  return (
    <>
      <PageHead
        eyebrow="Module 02"
        title="Symptom analysis"
        sub="Structured intake runs through the deterministic rules engine before the AI
is called. The result is a ranked list of possibilities – never a diagnosis."
      />

      <div className="mb-6 flex items-center gap-3">
        {steps.map((s, i) => (
          <div key={s} className="flex items-center gap-3">
            <button
              onClick={() => setStep(i)}

```

```

        className={`flex items-center gap-2 rounded-lg px-2.5 py-1.5 text-[13px]
font-medium transition ${
            i === step ? 'bg-brand-soft text-brand' : i < step ? 'text-ink hover:bg-
surface' : 'text-muted'
        }}
    >
    <span className={`grid size-5 place-items-center rounded-full text-[11px]
font-semibold ${
        i <= step ? 'bg-brand text-white' : 'bg-line text-muted'
    }`} >
        {i < step ? <Icon name="check" className="size-3" strokeWidth={3} /> : i +
1}

    </span>
    {s}
</button>
    {i < steps.length - 1 && <span className="h-px w-6 bg-line sm:w-10" />}
</div>
    )}}
</div>

<div className="grid gap-6 lg:grid-cols-[1.6fr_1fr] lg:items-start">
    <Card>
        {step === 0 && (
            <>
                <CardHead title="What are you experiencing?" sub="Select everything that
applies – you can refine next." />
                <div className="p-5">
                    <div className="flex flex-wrap gap-2">
                        {symptomOptions.map((s) => {
                            const on = selected.includes(s)
                            return (
                                <button
                                    key={s}
                                    onClick={() => toggle(s)}
                                    className={`rounded-lg px-3 py-1.5 text-[13px] font-medium
transition ${
                                        on
                                            ? 'bg-ink text-white'
                                            : 'bg-white text-slate ring-1 ring-line hover:ring-[#cfd8e3]'
                                        }`}
                                >
                                    {s}
                                </button>
                            )
                        })}
                    </div>
                </div>
            </>
        )}
    </div>

```

```

        <div className="mt-5">
          <Field label="Anything not listed above" hint="Free text is normalised
and redacted before it leaves the API layer.">
            <textarea rows={3} className={inputCls} placeholder="Describe in your
own words..." />
          </Field>
        </div>
      </div>
    </>
  )}

  {step === 1 && (
    <>
      <CardHead title="A little more context" sub="These fields sharpen the rules
engine, not just the prompt." />
      <div className="space-y-5 p-5">
        <div className="grid gap-5 sm:grid-cols-2">
          <Field label="How long has this been going on?">
            <select className={inputCls} defaultValue="3 days">
              {['Less than a day', '1-2 days', '3 days', 'About a week', 'More
than two weeks'].map((o) => (
                <option key={o}>{o}</option>
              ))}
            </select>
          </Field>
          <Field label="Highest recorded temperature">
            <input className={inputCls} defaultValue="38.1 °C" />
          </Field>
        </div>

        <div>
          <div className="mb-2 flex items-baseline justify-between">
            <span className="text-[13px] font-medium text-slate">Overall
severity</span>
            <span className="text-[13px] font-semibold text-ink">{severity} /
10</span>
          </div>
          <input
            type="range" min="1" max="10" value={severity}
            onChange={(e) => setSeverity(Number(e.target.value))}
            className="w-full accent-[#635bff]"
          />
          <div className="mt-1 flex justify-between text-[11.5px] text-muted">
            <span>Barely noticeable</span><span>Unbearable</span>
          </div>
        </div>
      </div>
    </>
  )}

```

```

    <Field label="Is it getting better or worse?">
      <div className="flex gap-2">
        {[ 'Improving', 'Stable', 'Worsening' ].map((o, i) => (
          <label key={o} className="flex-1">
            <input type="radio" name="trend" defaultChecked={i === 1}
className="peer sr-only" />
            <span className="block cursor-pointer rounded-lg py-2 text-center
text-[13px] font-medium text-slate ring-1 ring-line transition peer-checked:bg-brand-soft
peer-checked:text-brand peer-checked:ring-brand/30">
              {o}
            </span>
          </label>
        ))}
      </div>
    </Field>
  </div>
</>
)}

{step === 2 && (
  <>
    <CardHead
      title="Assessment"
      sub="Generated 18 Aug 2025 · session #4821"
      action={<Badge tone={triageResult.tone}>{triageResult.urgency}</Badge>}
    />
    <div className="p-5">
      <div className="flex items-start gap-3 rounded-lg bg-teal-soft p-4">
        <Icon name="shield" className="mt-0.5 size-[18px] shrink-0 text-teal" />
        <p className="text-[13px] leading-relaxed text-slate">
          <span className="font-semibold text-ink">No red flags
triggered.</span> The rules
          engine cleared this session before the AI was called. Urgency is
          routine –
            self-care with monitoring.
          </p>
        </div>

        <p className="mb-3 mt-6 text-[11.5px] font-semibold uppercase tracking-
[0.12em] text-muted">
          Ranked possibilities
        </p>
        <ul className="space-y-4">
          {triageResult.conditions.map((c, i) => (
            <li key={c.name}>
              <div className="mb-1.5 flex items-baseline justify-between gap-4">
                <span className="text-[14px] font-medium text-ink">{c.name}</span>

```

```

        <span className="text-[13px] font-semibold text-slate">
            {Math.round(c.likelihood * 100)}%
        </span>
    </div>
    <Meter value={c.likelihood} tone={i === 0 ? 'brand' : 'slate'} />
    <p className="mt-1.5 text-[12.5px] leading-relaxed text-
muted">{c.note}</p>
    </li>
    )}}
</ul>

    <p className="mb-3 mt-7 text-[11.5px] font-semibold uppercase tracking-
[0.12em] text-muted">
        Suggested next steps
    </p>
    <ul className="space-y-2.5">
        {triageResult.advice.map((a) => (
            <li key={a} className="flex gap-2.5 text-[13.5px] leading-relaxed
text-slate">
                <Icon name="check" className="mt-0.5 size-4 shrink-0 text-teal"
strokeWidth={2.2} />
                {a}
            </li>
        )}}
    </ul>

    <div className="mt-6 flex items-start gap-2.5 rounded-lg border border-
line bg-surface p-3.5">
        <Icon name="alert" className="mt-px size-4 shrink-0 text-amber" />
        <p className="text-[12.5px] leading-relaxed text-slate">
            This is decision support, not a diagnosis. MedIntel does not prescribe
medication
            and is not an emergency service. Consult a licensed physician.
        </p>
    </div>
</div>
</>
)}}

<div className="flex items-center justify-between border-t border-line px-5 py-
4">
    <Button variant="ghost" disabled={step === 0} onClick={() => setStep((s) => s
- 1)}>
        Back
    </Button>
    {step < 2 ? (
        <Button onClick={() => setStep((s) => s + 1)}>

```

```

        {step === 1 ? 'Run analysis' : 'Continue'} <Icon name="arrow"
className="size-4" />
      </Button>
    ) : (
      <Button as="link" to="/app/chat">
        Discuss in chat <Icon name="arrow" className="size-4" />
      </Button>
    )}
  </div>
</Card>

<div className="space-y-6">
  <Card>
    <CardHead title="Session summary" />
    <dl className="divide-y divide-line text-[13px]">
      {[
        ['Symptoms', selected.length ? `${selected.length} selected` : 'None
yet'],
        ['Severity', `${severity} / 10`],
        ['Red flags', '0 matched'],
        ['Confidence', `${Math.round(triageResult.confidence * 100)}%`],
      ].map(([k, v]) => (
        <div key={k} className="flex items-center justify-between px-5 py-3">
          <dt className="text-muted">{k}</dt>
          <dd className="font-medium text-ink">{v}</dd>
        </div>
      )))}
    </dl>
  </Card>

  <Card>
    <CardHead title="Pipeline trace" sub="What the request passed through" />
    <ol className="divide-y divide-line">
      {[
        'Normalise', 'Redact PII', 'Red-flag rules', 'Context assembly', 'Provider
call', 'Schema validation', 'Guardrail wrap'
      ].map((s, i) => (
        <li key={s} className="flex items-center gap-3 px-5 py-2.5">
          <span className={`grid size-5 shrink-0 place-items-center rounded-full
text-[10.5px] font-semibold ${
            step === 2 ? 'bg-teal-soft text-teal' : 'bg-surface text-muted ring-1
ring-line'
          }`}>
            {step === 2 ? <Icon name="check" className="size-3" strokeWidth={3} />
: i + 1}
          </span>
          <span className="text-[13px] text-slate">{s}</span>
          <span className="ml-auto text-[11.5px] text-muted">
            {step === 2 ? `${8 + i * 3} ms` : '-'}

```

```

        </span>
      </li>
    )}}
  </ol>
</Card>
</div>
</div>
</>
)
}

```

src/pages/Chat.jsx

```

import { useState, useRef, useEffect } from 'react'
import Icon from '../components/Icon'
import { Card, CardHead, Badge, Button, PageHead } from '../components/ui'
import { messages as seed, user } from '../data/mock'

const suggestions = [
  'Should I be worried about the night cough?',
  'How does my asthma change this?',
  'When should I see a doctor in person?',
]

export default function Chat() {
  const [messages, setMessages] = useState(seed)
  const [draft, setDraft] = useState('')
  const [typing, setTyping] = useState(false)
  const endRef = useRef(null)

  useEffect(() => {
    endRef.current?.scrollIntoView({ behavior: 'smooth', block: 'end' })
  }, [messages, typing])

  const send = (text) => {
    const body = (text ?? draft).trim()
    if (!body) return
    setMessages((m) => [...m, { from: 'user', text: body }])
    setDraft('')
    setTyping(true)
    setTimeout(() => {
      setTyping(false)
      setMessages((m) => [
        ...m,
        {

```



```

        from: 'bot',
        text: 'I have logged that against your 14 Aug session. Nothing you describe
matches a red-flag pattern, so this stays at routine urgency.\n\nKeep monitoring for
wheeze or breathlessness – with your asthma history those are the signals that would
change the assessment.',
    },
  ])
}, 900)
}

return (
  <>
    <PageHead
      eyebrow="Module 03"
      title="Health chat"
      sub="Follow-up conversation that carries your session context and stored history
into every turn – under the same guardrails as triage."
    />

    <div className="grid gap-6 lg:grid-cols-[1.7fr_1fr] lg:items-start">
      <Card className="flex h-[calc(100vh-13rem)] max-h-[620px] min-h-[440px] flex-col">
        <CardHead
          title="Session #4821"
          sub="Sore throat, fever · opened 14 Aug"
          action={<Badge tone="teal">Routine</Badge>}
        />

        <div className="flex-1 space-y-4 overflow-y-auto px-5 py-5">
          {messages.map((m, i) =>
            m.meta ? (
              <div key={i} className="flex items-start gap-2.5 rounded-lg border border-
line bg-surface p-3.5">
                <Icon name="alert" className="mt-px size-4 shrink-0 text-amber" />
                <p className="text-[12.5px] leading-relaxed text-slate">{m.text}</p>
              </div>
            ) : (
              <div key={i} className={`flex gap-3 ${m.from === 'user' ? 'justify-end' :
''}`>
                {m.from === 'bot' && (
                  <span className="mt-0.5 grid size-7 shrink-0 place-items-center
rounded-lg bg-ink text-white">
                    <Icon name="spark" className="size-3.5" />
                  </span>
                )}
              <div
                className={`max-w-[78%] whitespace-pre-line rounded-xl px-4 py-2.5
text-[13.5px] leading-relaxed ${

```

```

        m.from === 'user'
        ? 'bg-brand text-white'
        : 'bg-surface text-slate ring-1 ring-line'
      }` }
    >
    {m.text}
  </div>
  {m.from === 'user' && (
    <span className="mt-0.5 grid size-7 shrink-0 place-items-center
rounded-lg bg-surface text-[10.5px] font-semibold text-slate ring-1 ring-line">
      {user.initials}
    </span>
  )}
</div>
)
)}

{typing && (
  <div className="flex gap-3">
    <span className="mt-0.5 grid size-7 shrink-0 place-items-center rounded-lg
bg-ink text-white">
      <Icon name="spark" className="size-3.5" />
    </span>
    <div className="flex items-center gap-1 rounded-xl bg-surface px-4 py-3
ring-1 ring-line">
      {[0, 1, 2].map((i) => (
        <span
          key={i}
          className="size-1.5 animate-bounce rounded-full bg-muted"
          style={{ animationDelay: `${i * 120}ms` }}
        />
      ))}
    </div>
  </div>
)}
<div ref={endRef} />
</div>

<div className="border-t border-line p-4">
  <div className="mb-3 flex flex-wrap gap-2">
    {suggestions.map((s) => (
      <button
        key={s}
        onClick={() => send(s)}
        className="rounded-lg bg-white px-2.5 py-1.5 text-[12.5px] font-medium
text-slate ring-1 ring-line transition hover:text-ink hover:ring-[#cfd8e3]"
      >

```



```

        'PII redacted before the provider boundary',
        'Response schema-validated on return',
        'No drug names or dosages permitted',
        'Red-flag rules re-run on every turn',
      ].map((t) => (
        <li key={t} className="flex gap-2.5 text-[13px] leading-relaxed text-slate">
          <Icon name="check" className="mt-0.5 size-4 shrink-0 text-teal"
strokeWidth={2.2} />
          {t}
        </li>
      )))
    </ul>
  </Card>
</div>
</div>
</>
)
}

```

src/pages/Reports.jsx

```

import { useState } from 'react'
import Icon from '../components/Icon'
import { Card, CardHead, Badge, Button, PageHead } from '../components/ui'
import { reports } from '../data/mock'

const values = [
  { label: 'Haemoglobin', value: '14.2 g/dL', range: '13.0 - 17.0', ok: true },
  { label: 'WBC count', value: '11.4 ×109/L', range: '4.0 - 11.0', ok: false },
  { label: 'Platelets', value: '265 ×109/L', range: '150 - 410', ok: true },
  { label: 'ESR', value: '18 mm/hr', range: '0 - 22', ok: true },
]

export default function Reports() {
  const [drag, setDrag] = useState(false)

  return (
    <>
      <PageHead
        eyebrow="Module 04"
        title="Medical reports"
        sub="Upload a PDF or image. Extraction runs off the request path on a worker
queue, then attaches a plain-language summary to your timeline."
      >

```

```

    <Button><Icon name="upload" className="size-4" /> Upload report</Button>
  </PageHead>

  <div className="grid gap-6 lg:grid-cols-[1fr_1.25fr] lg:items-start">
    <div className="space-y-6">
      <div
        onDragOver={(e) => { e.preventDefault(); setDrag(true) }}
        onDragLeave={() => setDrag(false)}
        onDrop={(e) => { e.preventDefault(); setDrag(false) }}
        className={`flex flex-col items-center justify-center rounded-xl border
border-dashed px-6 py-12 text-center transition ${
          drag ? 'border-brand bg-brand-soft' : 'border-[#cfd8e3] bg-white'
        }}`
      >
        <span className="grid size-10 place-items-center rounded-lg bg-surface text-
slate ring-1 ring-line">
          <Icon name="upload" className="size-5" />
        </span>
        <p className="mt-4 text-[14px] font-medium text-ink">Drop a report here</p>
        <p className="mt-1 text-[12.5px] text-muted">PDF, JPG or PNG · up to 10 MB</p>
        <Button variant="secondary" size="sm" className="mt-4">Browse files</Button>
      </div>

      <Card>
        <CardHead title="On file" sub={` ${reports.length} reports`} />
        <ul className="divide-y divide-line">
          {reports.map((r) => (
            <li key={r.name} className="flex items-center gap-3 px-5 py-3.5
transition-colors hover:bg-surface">
              <span className="grid size-8 shrink-0 place-items-center rounded-lg bg-
surface text-slate ring-1 ring-line">
                <Icon name="file" className="size-4" />
              </span>
              <div className="min-w-0 flex-1">
                <p className="truncate text-[13.5px] font-medium text-
ink">{r.name}</p>
                <p className="text-[12px] text-muted">{r.lab} · {r.date}</p>
              </div>
              <Badge tone={r.tone}>{r.status}</Badge>
            </li>
          ))}
        </ul>
      </Card>
    </div>

    <Card>
      <CardHead

```

```

    title="Complete Blood Count"
    sub="Apollo Diagnostics · 12 Aug 2025"
    action={<Badge tone="amber">1 value flagged</Badge>}
  />

  <div className="p-5">
    <p className="mb-3 text-[11.5px] font-semibold uppercase tracking-[0.12em] text-muted">
      Extracted values
    </p>
    <div className="overflow-hidden rounded-lg ring-1 ring-line">
      <table className="w-full text-[13px]">
        <thead>
          <tr className="bg-surface text-left text-[11.5px] uppercase tracking-[0.08em] text-muted">
            <th className="px-4 py-2.5 font-semibold">Marker</th>
            <th className="px-4 py-2.5 font-semibold">Result</th>
            <th className="px-4 py-2.5 font-semibold">Reference</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-line">
          {values.map((v) => (
            <tr key={v.label} className="bg-white">
              <td className="px-4 py-2.5 text-slate">{v.label}</td>
              <td className={`px-4 py-2.5 font-medium ${v.ok ? 'text-ink' : 'text-amber'} `}>
                {v.value}
                {!v.ok && <span className="ml-1.5 text-[11px] font-semibold">HIGH</span>}
              </td>
              <td className="px-4 py-2.5 text-muted">{v.range}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>

    <p className="mb-2 mt-6 text-[11.5px] font-semibold uppercase tracking-[0.12em] text-muted">
      Summary
    </p>
    <p className="text-[13.5px] leading-relaxed text-slate">
      Most markers sit comfortably within the reference range. The white cell count is mildly elevated, which is a common finding during or shortly after an infection and
    </p>
  </div>

```

```

        consistent with the symptom session recorded on 14 August. It is worth
repeating once
        the current illness has resolved so the trend can be compared.
    </p>

    <div className="mt-5 flex items-start gap-2.5 rounded-lg border border-line
bg-surface p-3.5">
        <Icon name="alert" className="mt-px size-4 shrink-0 text-amber" />
        <p className="text-[12.5px] leading-relaxed text-slate">
            Summaries describe what a report contains. They do not interpret it
clinically, and
            they never recommend medication. Discuss flagged values with your
physician.
        </p>
    </div>

    <div className="mt-5 flex gap-2">
        <Button variant="secondary" size="sm">Download original</Button>
        <Button variant="ghost" size="sm">Add to timeline note</Button>
    </div>
</div>
</Card>
</div>
</>
)
}

```

src/pages/Reminders.jsx

```

import { useState } from 'react'
import Icon from '../components/Icon'
import { Card, CardHead, Badge, Button, Field, inputCls, PageHead } from
'../components/ui'
import { reminders } from '../data/mock'

const week = ['M', 'T', 'W', 'T', 'F', 'S', 'S']
const log = [
  [1, 1, 1, 1, 1, 0, 1],
  [1, 1, 0, 1, 1, 1, 1],
  [1, 1, 1, 1, 1, 1, 1],
  [1, 0, 1, 1, 1, 1, 2],
]

export default function Reminders() {

```

```

const [items, setItems] = useState(reminders.map((r) => ({ ...r, on: true })))

const toggle = (i) =>
  setItems((prev) => prev.map((r, idx) => (idx === i ? { ...r, on: !r.on } : r)))

return (
  <>
    <PageHead
      eyebrow="Module 05"
      title="Medicine reminders"
      sub="Schedules run as recurring jobs off the request path. Every dose is logged as
taken or missed, which is what the adherence figure is built from."
    >
      <Button><Icon name="plus" className="size-4" /> Add schedule</Button>
    </PageHead>

    <div className="grid gap-6 lg:grid-cols-[1.5fr_1fr] lg:items-start">
      <div className="space-y-6">
        <Card>
          <CardHead title="Active schedules" sub={`\${items.filter((i) => i.on).length}
of \${items.length} enabled`} />
          <ul className="divide-y divide-line">
            {items.map((r, i) => (
              <li key={r.drug} className="flex items-center gap-4 px-5 py-4">
                <span className="grid size-9 shrink-0 place-items-center rounded-lg bg-
surface text-slate ring-1 ring-line">
                  <Icon name="clock" className="size-[18px]" />
                </span>

                <div className="min-w-0 flex-1">
                  <p className="truncate text-[14px] font-medium text-ink">{r.drug}</p>
                  <p className="mt-0.5 text-[12.5px] text-muted">
                    {r.time} · {r.freq} · next {r.next.toLowerCase()}
                  </p>
                </div>

                <div className="hidden text-right sm:block">
                  <p className={`text-[13px] font-semibold \${r.tone === 'amber' ? 'text-
amber' : 'text-teal'}`}>
                    {Math.round(r.adherence * 100)}%
                  </p>
                  <p className="text-[11.5px] text-muted">adherence</p>
                </div>

                <button
                  onClick={() => toggle(i)}
                  role="switch"

```



```

        </span>
      )}}
    </div>
  </div>
</Card>
</div>

<div className="space-y-6">
  <Card>
    <CardHead title="New schedule" sub="Fields only – nothing is dispensed or
prescribed" />
    <div className="space-y-4 p-5">
      <Field label="Medication name">
        <input className={inputCls} placeholder="As written on your prescription"
/>

      </Field>
      <div className="grid grid-cols-2 gap-4">
        <Field label="Dose">
          <input className={inputCls} placeholder="e.g. 10 mg" />
        </Field>
        <Field label="Time">
          <input type="time" defaultValue="21:00" className={inputCls} />
        </Field>
      </div>
      <Field label="Repeat">
        <select className={inputCls}>
          {[ 'Daily', 'Twice daily', 'Weekly', 'Every other day', 'Custom' ].map((o)
=> (
            <option key={o}>{o}</option>
          ))}
        </select>
      </Field>
      <Field label="Notify by" hint="Delivery is queued, not sent inline with the
request.">
        <div className="flex gap-2">
          {[ 'Push', 'E-mail', 'Both' ].map((o, i) => (
            <label key={o} className="flex-1">
              <input type="radio" name="notify" defaultChecked={i === 2}
className="peer sr-only" />
              <span className="block cursor-pointer rounded-lg py-2 text-center
text-[13px] font-medium text-slate ring-1 ring-line transition peer-checked:bg-brand-soft
peer-checked:text-brand peer-checked:ring-brand/30">
                {o}
              </span>
            </label>
          ))}
        </div>
      </Field>
    </div>
  </Card>
</div>

```

```

        </Field>
        <Button className="w-full">Create schedule</Button>
      </div>
    </Card>

    <Card>
      <CardHead title="Next 24 hours" />
      <ul className="divide-y divide-line">
        {[['21:00', 'Montelukast 10 mg', 'Tonight'], ['08:00', 'Azithromycin 500
mg', 'Tomorrow']].map(([t, d, w]) => (
          <li key={d} className="flex items-center gap-3 px-5 py-3">
            <span className="w-11 text-[12.5px] font-semibold text-ink">{t}</span>
            <span className="flex-1 truncate text-[13px] text-slate">{d}</span>
            <Badge tone="slate">{w}</Badge>
          </li>
        ))}
      </ul>
    </Card>
  </div>
</div>
</>
)
}

```

src/pages/History.jsx

```

import { useState } from 'react'
import Icon from '../components/Icon'
import { Card, CardHead, Badge, Button, PageHead, Empty } from '../components/ui'
import { timeline } from '../data/mock'

const filters = ['All', 'Triage', 'Chat', 'Report', 'Medication']

export default function History() {
  const [filter, setFilter] = useState('All')
  const shown = filter === 'All' ? timeline : timeline.filter((t) => t.kind === filter)

  return (
    <>
      <PageHead
        eyebrow="Module 06"
        title="Medical history"
        sub="One chronological record of every session, report and medication change –
assembled for a real consultation, not for legal or insurance use."
      >

```

```

    <Button variant="secondary">Export as PDF</Button>
  </PageHead>

  <div className="mb-6 flex flex-wrap items-center gap-2">
    {filters.map((f) => (
      <button
        key={f}
        onClick={() => setFilter(f)}
        className={`rounded-lg px-3 py-1.5 text-[13px] font-medium transition ${
          filter === f ? 'bg-ink text-white' : 'bg-white text-slate ring-1 ring-line
hover:ring-[#cfd8e3]'
        }`}
      >
        {f}
      </button>
    ))}
    <span className="ml-auto text-[12.5px] text-muted">{shown.length} entries</span>
  </div>

  <div className="grid gap-6 lg:grid-cols-[1.7fr_1fr] lg:items-start">
    <Card className="overflow-hidden">
      {shown.length === 0 ? (
        <Empty
          title="Nothing recorded yet"
          sub="Entries appear here as you complete sessions, upload reports or change
medication."
          action={<Button as="link" to="/app/symptoms" size="sm">Start a
session</Button>}
        />
      ) : (
        <ol className="relative px-5 py-5">
          <span className="absolute left-[38px] top-8 bottom-8 w-px bg-line" />
          {shown.map((t) => (
            <li key={t.title} className="relative flex gap-4 pb-6 last:pb-0">
              <span className="relative z-10 mt-0.5 grid size-6 shrink-0 place-items-
center rounded-full bg-white ring-1 ring-line">
                <span className={`size-2 rounded-full ${
                  { brand: 'bg-brand', teal: 'bg-teal', amber: 'bg-amber', rose: 'bg-
rose' }[t.tone]
                }`} />
              </span>
              <div className="min-w-0 flex-1 rounded-lg border border-line p-4
transition hover:border-[#cfd8e3]">
                <div className="flex flex-wrap items-center gap-2">
                  <Badge tone={t.tone}>{t.kind}</Badge>
                  <span className="text-[12px] text-muted">{t.date}</span>
                </div>

```

```

        <p className="mt-2 text-[14px] font-medium text-ink">{t.title}</p>
        <p className="mt-1 text-[13px] leading-relaxed text-
slate">{t.body}</p>
      </div>
    </li>
  )})
</ol>
})
</Card>

<div className="space-y-6">
  <Card>
    <CardHead title="At a glance" sub="Rolling 12 months" />
    <dl className="divide-y divide-line text-[13px]">
      {[
        ['Symptom sessions', '7'],
        ['Escalations triggered', '0'],
        ['Reports uploaded', '3'],
        ['Medication changes', '4'],
        ['Average adherence', '87%'],
      ].map(([k, v]) => (
        <div key={k} className="flex items-center justify-between px-5 py-3">
          <dt className="text-muted">{k}</dt>
          <dd className="font-medium text-ink">{v}</dd>
        </div>
      ))}
    </dl>
  </Card>

  <Card>
    <CardHead title="Export" sub="What a physician receives" />
    <ul className="space-y-3 p-5">
      {[
        'Chronological session and report list',
        'Profile: age, sex, allergies, conditions',
        'Current medication and adherence figures',
        'Every AI output marked as decision support',
      ].map((t) => (
        <li key={t} className="flex gap-2.5 text-[13px] leading-relaxed text-
slate">
          <Icon name="check" className="mt-0.5 size-4 shrink-0 text-teal"
strokeWidth={2.2} />
          {t}
        </li>
      ))}
    </ul>
  </Card>

```

```

        </div>
      </div>
    </>
  )
}

```

src/pages/Profile.jsx

```

import Icon from '../components/Icon'
import { Card, CardHead, Badge, Button, Field, inputCls, PageHead } from
'../components/ui'
import { user } from '../data/mock'

export default function Profile() {
  return (
    <>
      <PageHead
        eyebrow="Module 01"
        title="Profile & account"
        sub="Everything here is injected as context before the AI is called, and every
field is redacted at the provider boundary."
      >
        <Button>Save changes</Button>
      </PageHead>

      <div className="grid gap-6 lg:grid-cols-[1.4fr_1fr] lg:items-start">
        <div className="space-y-6">
          <Card>
            <CardHead title="Personal details" />
            <div className="space-y-4 p-5">
              <div className="flex items-center gap-4">
                <span className="grid size-14 place-items-center rounded-full bg-ink text-
[16px] font-semibold text-white">
                  {user.initials}
                </span>
                <div>
                  <p className="text-[15px] font-semibold text-ink">{user.name}</p>
                  <p className="text-[13px] text-muted">{user.email}</p>
                </div>
                <Button variant="secondary" size="sm" className="ml-auto">Change
photo</Button>
              </div>

              <div className="grid gap-4 sm:grid-cols-2">
                <Field label="Full name">

```

```

        <input className={inputCls} defaultValue={user.name} />
      </Field>
      <Field label="Email">
        <input className={inputCls} defaultValue={user.email} />
      </Field>
      <Field label="Age">
        <input className={inputCls} defaultValue={user.age} />
      </Field>
      <Field label="Sex">
        <select className={inputCls} defaultValue={user.sex}>
          {[ 'Male', 'Female', 'Other', 'Prefer not to say' ].map((o) => <option
key={o}>{o}</option>)}
        </select>
      </Field>
    </div>
  </div>
</Card>

<Card>
  <CardHead title="Clinical background" sub="Raises or lowers baseline
probability during triage" />
  <div className="space-y-5 p-5">
    <div>
      <p className="mb-2 text-[13px] font-medium text-slate">Allergies</p>
      <div className="flex flex-wrap gap-2">
        {user.allergies.map((a) => (
          <Badge key={a} tone="rose">{a}</Badge>
        ))}
        <button className="rounded-md px-2 py-0.5 text-[11.5px] font-semibold
text-brand ring-1 ring-line hover:bg-brand-soft">
          + Add
        </button>
      </div>
    </div>
    <div>
      <p className="mb-2 text-[13px] font-medium text-slate">Chronic
conditions</p>
      <div className="flex flex-wrap gap-2">
        {user.conditions.map((c) => (
          <Badge key={c} tone="amber">{c}</Badge>
        ))}
        <button className="rounded-md px-2 py-0.5 text-[11.5px] font-semibold
text-brand ring-1 ring-line hover:bg-brand-soft">
          + Add
        </button>
      </div>
    </div>
  </div>
</Card>

```

```

    <Field label="Notes for the clinician" hint="Free text is normalised and
redacted before it leaves the API layer.">
      <textarea rows={3} className={inputCls} defaultValue="Inhaler used
seasonally, roughly twice a year." />
    </Field>
  </div>
</Card>
</div>

<div className="space-y-6">
  <Card>
    <CardHead title="Security" />
    <ul className="divide-y divide-line">
      {[
        ['Password', 'Updated 3 months ago', 'Change'],
        ['Two-factor auth', 'Not enabled', 'Enable'],
        ['Active sessions', '2 devices', 'Review'],
      ]}.map(([k, v, a]) => (
        <li key={k} className="flex items-center gap-3 px-5 py-3.5">
          <div className="min-w-0 flex-1">
            <p className="text-[13.5px] font-medium text-ink">{k}</p>
            <p className="text-[12px] text-muted">{v}</p>
          </div>
          <Button variant="secondary" size="sm">{a}</Button>
        </li>
      )))
    </ul>
  </Card>

  <Card>
    <CardHead title="Data & privacy" />
    <ul className="space-y-3 p-5">
      {[
        'Identifiers stripped before any AI request',
        'Encrypted in transit and at rest',
        'Per-resource ownership checks on every read',
        'Append-only audit log of access',
      ]}.map((t) => (
        <li key={t} className="flex gap-2.5 text-[13px] leading-relaxed text-
slate">
          <Icon name="shield" className="mt-0.5 size-4 shrink-0 text-teal" />
          {t}
        </li>
      )))
    </ul>
    <div className="border-t border-line px-5 py-4">
      <button className="text-[13px] font-medium text-rose hover:underline">

```



```
        Delete account and all records
      </button>
    </div>
  </Card>
</div>
</div>
</>
)
}
```